Activity 1: Current Working Directory

It is often useful to check our current working directory before beginning to work with files so that we can provide appropriate file paths. We can do this using the method getcwd of the module os which displays the file path of the current working directory.

```
import os
path = os.getcwd()
print(f"Current Working Directory: {path}")
```

We can also display the content of this directory using the method listdir of the module os as follows:

```
for file in os.listdir(path):
  print(file)
```

[Tasks]

We wish to create a program to display information about the current working directory.

Start by creating a new folder called week6, inside of which you should create a python file called txt_file_tasks.py.

The program should consist of the following two functions:

The first function should be named cwd and should have no parameters.
The function should retrieve the file path for the current working folder.
The function should then display the message "The current working directory is {path}" where {path} is the previously retrieved path.
The function should then display the message "The directory contains the following files:".
Finally, the function should display each file contained in the current working directory.

The second function should be named run and should have no parameters.
The function should display the message "Processing...".
The function should then call the first function.

An example run of such a program is shown below:

```
Processing...
The current working directory is com411/week6
The directory contains the following files:
txt_file_tasks.py
```

Code your solution and be sure to include appropriate comments in your code.

?

[Commit to Repository]

It is time to commit our working program to our repository. Use suitable commands to commit your changes. You should add a description for the commit summarising the changes you have made e.g. "Added code to display information about the current working directory.".

Activity 2: Reading a File

We can read a file by simply providing a file path, absolute or relative, to the function open. For example, if we wish to read the file named data.txt that is stored in the current working directory then we can do the following:

```
file = open("data.txt")
```

The above code will open the file data.txt relative to the current working directory and assign a reference to the opened file to the variable file.

It is important to close a file once we are done processing it so as to free it for use by other processes. We can do this simply by using the method close:

```
file.close()
```

We can also use the keyword with in order to automatically close a file once we are done processing it:

```
with open("data.txt") as file:
  # do something with the file.
  # the file will be closed at the end of the 'with' code block.
```

We can read the entire content of the file as a string using the method read. This will read in all the data including any white space and/or end of line (\n) characters. We can use the method split of string to divide the string into separate lines and remove any end of line (\n) characters.

```
with open("data.txt") as file:
  data = file.read()
  lines = data.split('\n')
```

We can also read only a few characters by supplying an appropriate value to the method read:

```
with open("data.txt") as file:
  partial_data = file.read(10)
```

We can also read a single line using the method readline. This will include the end of line character (\n) in the string. We can use the method stripof string to remove any leading or trailing characters. By default, it will remove any spaces including the new line character.

```
with open("data.txt") as file:
  line = file.readline().strip()
```

We can also specify what characters should be removed from the beginning or end of a string by supplying these a parameter to the method strip:

```
line = ",this is some sample text"

# remove leading and trailing commas
line.strip(",") # gives "this is some sample text"

# remove leading and trailing ",t"
line.strip(",t") # gives "his is some sample text"
```

[Tasks]

We wish to create a program to read a text file containing sections of a library.

You should start by creating a suitable text file named library.txt and store it in the following location:

```
week6/library.txt
```

The text file should contain the following text:

```
The Law Section
The Art Section
The Technology Section
The History Section
```

The program should consist of the following functions:

The first function should be named display_chars and should take 2 parameters which represent a file path and the number of characters to be read.
The function should start by opening the specified file for reading.
The function should then read in and display the specified number of characters.
Finally, the function should close the file if necessary.

The second function should be named display_line and should take 1 parameter which represents a file path.
The function should start by opening the specified file for reading.
The function should then read in and display the first line in the file.
Finally, the function should close the file if necessary.

The third function should be named display_text and should take 1 parameter which represents a file path.
The function should start by opening the specified file for reading.
The function should then read in and display all the text contained in the file.
Finally, the function should close the file if necessary.

The fourth function should be named run_task2 and should take no parameters.
The function should call each of the earlier functions with the value "library.txt" passed as the file path.

Finally, you should add the following code to the end of your file so that it executes the function run_task2 when the file is executed directly.

```
if __name__ == "__main__":
    run_task2()
```

An example run of such a program is shown below:

```
The first 5 characters are:
The L

The first line is:
The Law Section

The full text is:
The Law Section
The Art Section
The Technology Section
The History Section
```

Code your solution and include any appropriate comments in your code.

[Commit to Repository]

It is time to commit our working program to our repository. Use appropriate commands to commit your changes. You should add a description for the commit summarising the changes you have made e.g. "Added code to demonstrate how to read a text file.".

Activity 3: Reading a File

It is often useful to read a file line by line so that we can process each line individually. We can use a loop to do this.

```
with open("data.txt") as file:
  for line in file:
    # do something with the line from the file
    # line.strip() will remove any white space including the '\n' character
```

[Tasks]

We wish to create a program to iterate through the library.

The program should consist of the following two functions:

The first function should be named search and should take 1 parameter that represents a file name.
The function should start by displaying the message "Searching...".
The function should then open the specified file for reading.
For each line in the file:
  The function should display the message "Looked in {location}" where {location} is a line from the file.
The function should display the message "...Done!".
The function should close the file if necessary.

The second function should be named run_task3 and should have no parameters.
The function should call the first function with the path to the file library.txt.

An example run of such a program is shown below:

```
Searching...
Looked in The Law Section.
Looked in The Art Section
Looked in The Technology Section.
Looked in The History Section.
...Done!
```

Code your solution and be sure to include appropriate comments in your code.

[Commit to Repository]

It is time to commit our working program to our repository. Use appropriate commands to commit your changes. You should add a description for the commit summarising the changes you have made e.g. "Added code to read a text file line by line.".

Activity 4: Writing to a File

We can write to a file by opening a file in write mode and using the method write.

```
with open("output.txt", "w") as file:
  file.write("some text")
```

If the file does not already exist then a new file will be created otherwise the existing file will overwritten.

We can write multiple lines to a file by simply including the new line character (\n) in the text:

```
lines = "first line\nsecond line\n"
with open("output.txt", "w") as file:
  file.write(lines)
```

[Tasks]

We wish to create a program to sort the books.

You should start by creating a suitable text file named books.txt and store it in the following location:

```
week6/books.txt
```

The text file should contain the following:

```
Section: The Law Section
The Magna Carta
Section: The Technology Section
How to build a robot
How to build a flying car
Section: The History Section
The history of everything
```

The program should consist of the following three functions:

The first function should be named search_books and should take 1 parameter that represents a file path.
The function should start by displaying the message "Searching...".
The function should then create a string variable named sections and set its value to "" (i.e. an empty string)
The function should then create a string variable named books and set its value to "Books:\n".
The function should then open the specified file for reading.
For each line in the file, the function should do the following:
  If the line starts with "Section" then
    Add the line, including a new line character (\n), to the string sections.
  If the line does not start with "Section" then
    Add the line, including a new line character (\n), to the string books.
The function should close the file, if necessary.
The function should then display the message "Done!".
Finally, the function should return a single string consisting of sections and books and two new line characters between them i.e. the returned string should be in the format "{sections}\n\n{books}".

The second function should be named save and should take 2 parameters.
The first parameter is a file path.
The second parameter is a string that represents the data to be stored.
The function should display the message "Saving...".
The function should then open the specified file for writing.
The function should then write the data to the file.
The function should display the message "Done!".

The third function should be named run_task4 and should have no parameters.
The function should call the first function with the file path "books.txt" and store the value returned by the function in a local variable named data.
The function should then pass the variable as an argument to the second function along with the filename "section-books.txt".

An example run of such a program is shown below:

```
Searching...Done!
Saving...Done
```

The content of the file section-books.txt should be as follows:

```
Section: The Law Section
Section: The Technology Section
Section: The History Section

Books:
The Magna Carta
How to build a robot
How to build a flying car
The history of everything
```

Code your solution and be sure to include appropriate comments in your code.

[Commit to Repository]

It is time to commit our working program to our repository. Use appropriate commands to commit your changes. You should add a description for the commit summarising the changes you have made e.g. "Added code to demonstrate how to write to a text file.".

Originally created by Prins Butt, Updated and modified by Darren Cunningham.

Last modified: Saturday, 15 November 2025, 2:27 PM

◀ 1 - Data Files

Jump to...

3 - CSV Files ▶

Dashboard / Problem Solving Through Programming (QHO444) / QHO444 Materials / 2 - Text Files

Study

Succeed@Solent
Referencing
Subject Guides
Library
Ethics

Organise

Email
Timetables
Term Dates
Portal

Support

Student Hub
Technical Support
Printing
Extenuating Circumstances

Solent Futures

Solent Careers Online
CV Help
Placements

Disclaimer | Terms & Conditions | Cookies | Accessibility Statement |

© Solent University